

Sistema de Gestión de Biblioteca Universitaria

Documento de Especificación del Proyecto

Asignatura	Lenguaje de Programación 2
Docente	Ing. Bryan Vélez M.Sc.
Grupo	#1
Semestre	Tercero — Modalidad en Línea
Universidad	Universidad Agraria del Ecuador (UAE)
Fecha	Julio 2026

Índice

1. [Introducción](#)
 2. [Justificación](#)
 3. [Objetivos](#)
 4. [Alcance](#)
 5. [Marco Teórico](#)
 6. [Metodología de Desarrollo](#)
 7. [Diseño Preliminar](#)
 8. [Distribución del Trabajo](#)
 9. [Repositorio y Control de Versiones](#)
 10. [Plan de Trabajo y Cronograma](#)
-

1. Introducción

Las bibliotecas universitarias enfrentan el desafío de gestionar eficientemente sus recursos bibliográficos: registro de socios, control de préstamos, devoluciones y reservas. Actualmente, muchas bibliotecas pequeñas y medianas aún utilizan métodos manuales o sistemas genéricos que no se adaptan a sus necesidades específicas.

El presente proyecto tiene como objetivo desarrollar un **Sistema de Gestión de Biblioteca Universitaria** que automatice los procesos diarios de una biblioteca física. El sistema permitirá registrar socios (estudiantes y docentes), administrar el catálogo de libros, controlar préstamos y devoluciones, y gestionar reservas de ejemplares no disponibles.

El sistema se desarrolla como proyecto final de la asignatura **Lenguaje de Programación 2**, aplicando los conceptos de Programación Orientada a

Objetos, estructuras de datos, algoritmos de ordenamiento y búsqueda, bases de datos SQLite e interfaz gráfica con Tkinter, todos enseñados durante el semestre por el Ing. Bryan Vélez.

2. Justificación

La Universidad Agraria del Ecuador, en su modalidad en línea, requiere que los estudiantes demuestren competencias en el desarrollo de software aplicando los principios de la programación estructurada y orientada a objetos. Este proyecto integra los conocimientos adquiridos durante el semestre en un producto funcional.

La automatización de una biblioteca universitaria permite:

- **Reducir errores humanos** en el registro manual de préstamos y devoluciones
- **Agilizar la búsqueda** de libros en el catálogo
- **Mantener un control preciso** de los ejemplares disponibles
- **Mejorar la experiencia** del usuario (socios y bibliotecarios)

Además, el proyecto permite aplicar conceptos fundamentales de la ingeniería de software como el modelo incremental, el control de versiones con Git/GitHub, y el uso de documentación técnica.

3. Objetivos

3.1 Objetivo General

Desarrollar un sistema de escritorio para la gestión de una biblioteca universitaria que permita administrar socios, libros, préstamos y reservas, aplicando los principios de la Programación Orientada a Objetos, estructuras de datos, algoritmos de búsqueda y ordenamiento, y persistencia en base de datos SQLite.

3.2 Objetivos Específicos

1. Implementar un modelo de clases utilizando herencia, encapsulamiento, polimorfismo y abstracción para representar los actores del sistema (Persona → Estudiante/Docente, Libro, Préstamo, Reserva).
2. Utilizar estructuras de datos lineales (lista enlazada, cola FIFO, pila LIFO) para la gestión dinámica de la información en memoria.
3. Implementar algoritmos de ordenamiento (burbuja, inserción, merge sort, quick sort) y búsqueda (lineal, binaria) para la manipulación del catálogo.

4. Persistir los datos en una base de datos SQLite con operaciones CRUD completas y claves foráneas.
 5. Desarrollar una interfaz gráfica de usuario (GUI) utilizando Tkinter para la interacción con el sistema.
-

4. Alcance

4.1 Funcionalidades incluidas

- **Gestión de Socios:** registro, consulta y actualización de datos de estudiantes y docentes
- **Gestión de Libros:** alta, consulta y eliminación de libros en el catálogo
- **Préstamos:** realización de préstamos y registro de devoluciones
- **Reservas:** creación de reservas para libros no disponibles (cola de espera FIFO)
- **Búsqueda:** búsqueda de libros por título, autor, y aplicación de búsqueda binaria
- **Ordenamiento:** ordenamiento del catálogo por diferentes criterios usando múltiples algoritmos

4.2 Funcionalidades excluidas

- No incluye módulo de multas o sanciones
- No incluye generación de reportes estadísticos avanzados
- No incluye integración con sistemas externos (API web, RFID, etc.)

4.3 Usuarios del sistema

- **Bibliotecario:** usuario principal que opera el sistema para todas las funcionalidades
 - **Socio:** estudiante o docente cuyos datos son gestionados en el sistema
-

5. Marco Teórico

5.1 Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en clases y objetos. Los cuatro pilares fundamentales son:

- **Encapsulamiento:** ocultación de datos mediante atributos privados (self._atributo) y acceso controlado por getters/setters. En el sistema, todos los atributos de las clases del modelo son privados.
- **Herencia:** una clase puede heredar atributos y métodos de otra clase padre. Estudiante y Docente heredan de Persona.

- **Polimorfismo:** el mismo método puede comportarse de manera diferente según la clase que lo implementa. `tipo_socio()` retorna "Estudiante" o "Docente" según la subclase.
- **Abstracción:** se definen interfaces o métodos abstractos que las subclases deben implementar. `Persona.tipo_socio()` lanza `NotImplementedError` como contrato.

Referencia: Joyanes Aguilar, L. (2020). *Programación orientada a objetos con Python*. McGraw-Hill Interamericana.

5.2 Estructuras de Datos

5.2.1 Lista Enlazada Simple

Estructura lineal donde cada elemento (nodo) contiene un valor y un puntero al siguiente nodo. Permite inserciones y eliminaciones sin reorganizar memoria. Se implementó `LinkedList` como base para la cola y la pila.

Referencia: Weiss, M. A. (2013). *Estructuras de datos y algoritmos* (4ª ed.). Pearson Educación.

5.2.2 Cola (Queue — FIFO)

El primer elemento en entrar es el primero en salir (First In, First Out). Se utiliza para gestionar las reservas de libros en orden de llegada.

5.2.3 Pila (Stack — LIFO)

El último elemento en entrar es el primero en salir (Last In, First Out). Se utiliza para deshacer operaciones (undo) de préstamos y devoluciones.

Referencia: Cairo, O., & Guardati, S. (2006). *Estructuras de datos* (3ª ed.). McGraw-Hill.

5.3 Algoritmos de Ordenamiento y Búsqueda

5.3.1 Algoritmos de Ordenamiento

Algoritmo	Complejidad	Descripción
Bubble Sort	$O(n^2)$	Compara pares adyacentes y los intercambia si están en orden incorrecto
Insertion Sort	$O(n^2)$	Inserta cada elemento en su posición correcta dentro de la parte ordenada
Merge Sort	$O(n \log n)$	Divide la lista en

Algoritmo	Complejidad	Descripción
		mitades, las ordena recursivamente y las fusiona
Quick Sort	$O(n \log n)$ promedio	Selecciona un pivote y particiona en menores y mayores

5.3.2 Algoritmos de Búsqueda

Algoritmo	Complejidad	Requisito
Búsqueda Lineal	$O(n)$	Ninguno
Búsqueda Binaria	$O(\log n)$	Lista ordenada previamente

Referencia: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

5.4 Base de Datos SQLite

SQLite es un motor de base de datos embebido que no requiere servidor, ideal para aplicaciones de escritorio. El sistema utiliza:

- `sqlite3.connect()` para establecer conexión
- `cursor.execute()` con parámetros posicionales (?, ?) para consultas seguras
- `PRAGMA foreign_keys = ON` para activar claves foráneas
- Tablas relacionadas: socios, libros, prestamos (FK a socio y libro), reservas (FK a socio y libro)

Referencia: Kreibich, J. A. (2010). *Using SQLite*. O'Reilly Media.

5.5 Interfaz Gráfica con Tkinter

Tkinter es la biblioteca gráfica estándar de Python, incluida por defecto sin necesidad de instalación adicional. Se utiliza para construir la ventana principal, menús desplegados, formularios y cuadros de diálogo.

Referencia: Grayson, J. E. (2000). *Python and Tkinter programming*. Manning Publications.

6. Metodología de Desarrollo

Se utiliza el **Modelo Incremental**, que consiste en desarrollar el sistema por etapas o incrementos, donde cada incremento agrega funcionalidad completa al producto. Este modelo permite:

- Entregar versiones funcionales tempranas
- Obtener retroalimentación continua
- Reducir riesgos de integración
- Adaptarse a cambios durante el desarrollo

6.1 Incrementos del Proyecto

Incremento	Semana	Entregable	Estado
Incremento 1	Semana 12 (7-10 jul)	Modelo de clases POO + Estructuras de datos + Algoritmos + Esquema BD + GUI (stubs) + Documento de Especificación (50%)	☒ Completado
Incremento 2	Semana 13 (13-17 jul)	GUI funcional completa + BD poblada + CRUD completo + Pruebas unitarias + Manual de Usuario	☐ Pendiente
Incremento 3	Semana 14 (20 jul)	Integración final + Pruebas de sistema + Documento completo (100%) + Defensa oral	☐ Pendiente

6.2 Flujo de trabajo

1. **Planificación del incremento:** se definen las funcionalidades a implementar
2. **Diseño:** se actualizan los diagramas si es necesario
3. **Implementación:** cada miembro desarrolla su módulo asignado
4. **Pruebas:** se ejecutan pruebas unitarias y de integración
5. **Revisión:** el líder (Henry) revisa e integra los cambios
6. **Entrega:** se sube a GitHub y se documenta el avance

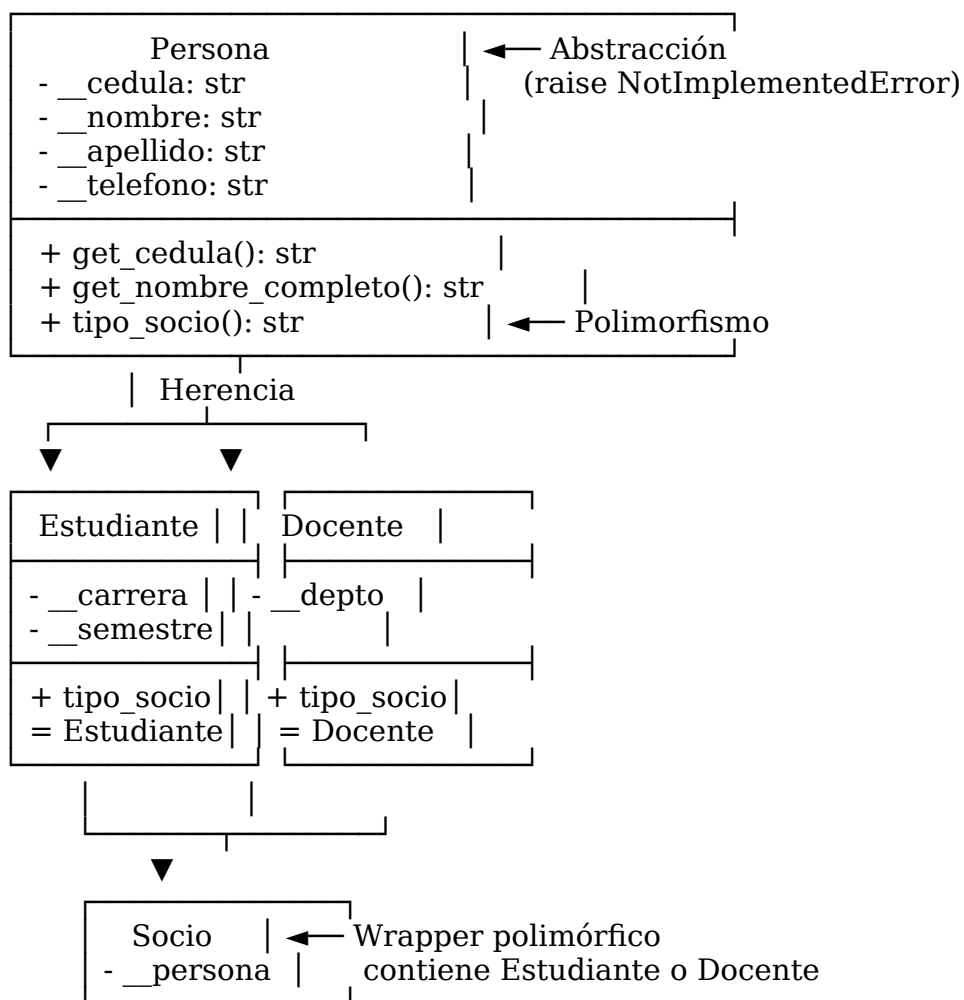
6.3 Herramientas de desarrollo

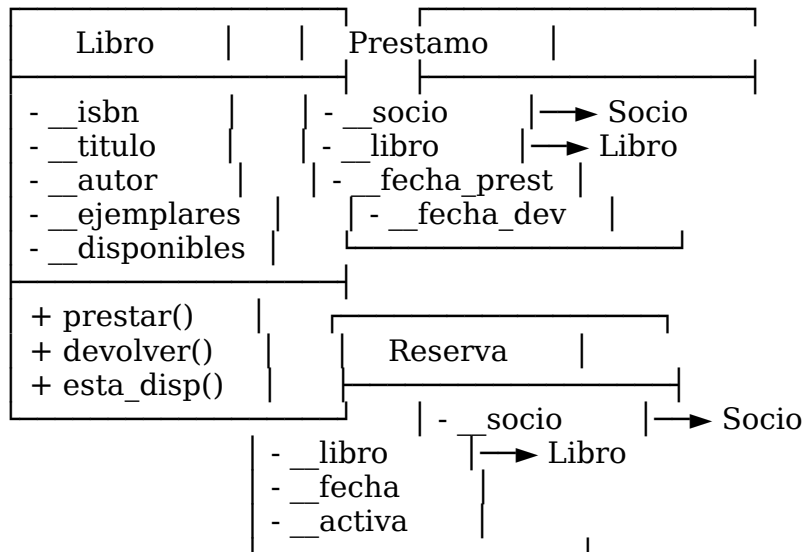
Herramienta	Versión	Propósito
Python	3.11+	Lenguaje de programación
Visual Studio Code	Última	Editor de código con extensión Python

Herramienta	Versión	Propósito
Tkinter	8.6	(Microsoft) y GitLens Biblioteca de interfaz gráfica (incluida en Python)
SQLite3	3.x	Motor de base de datos embebido
DB Browser for SQLite	3.x	Administración visual de la BD
Git	2.47+	Control de versiones
GitHub	—	Repositorio remoto

7. Diseño Preliminar

7.1 Diagrama de Clases (UML)





Estructuras de Datos:

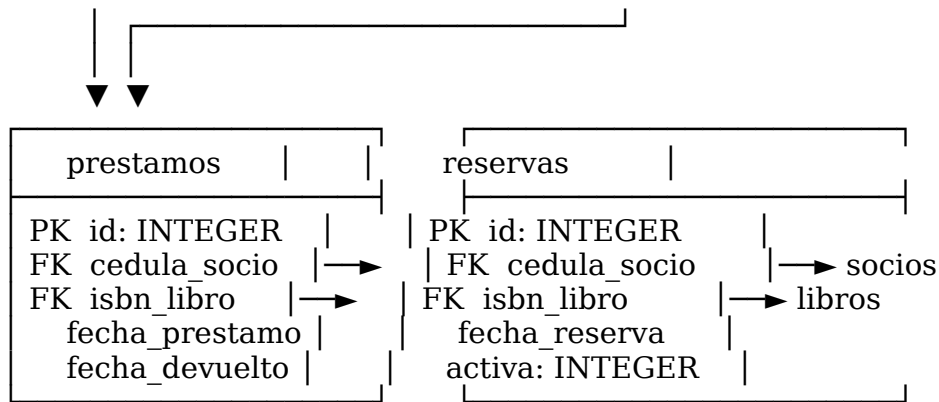
LinkedList (Nodo → Nodo)	Queue (FIFO)	Stack (LIFO)
append()	enqueue()	push()
prepend()	dequeue()	pop()
remove()	peek()	peek()
find()		

Algoritmos:

Búsqueda	Ordenamiento
Lineal ($O(n)$)	Bubble ($O(n^2)$)
Binaria ($O(\log n)$)	Insertion ($O(n^2)$)
	Merge ($O(n \log n)$)
	Quick ($O(n \log n)$)

7.2 Modelo Entidad-Relación (Base de Datos)

socios	libros
PK cedula: TEXT nombre: TEXT apellido: TEXT tipo: TEXT carrera: TEXT semestre: INT telefono: TEXT	PK isbn: TEXT titulo: TEXT autor: TEXT editorial: TEXT anio: INTEGER ejemplares: INT disponibles: INT



8. Distribución del Trabajo

8.1 Roles del equipo

Integrante	Rol	Módulo	Archivos
Henry Pazmiño	Líder / Integrador	Modelos POO + Integración	main.py, src/models/*, coordinación general
Jodie Parrales	Desarrolladora GUI	Interfaz gráfica Tkinter	src/gui/app.py
Cindy Ayoví	Desarrolladora BD	Base de datos SQLite	src/database/*
Cesar Gonzales	Desarrollador Lógica	Algoritmos + Estructuras	src/algorithms/*, src/data_structures/*
Mayra Vera	Documentación y Pruebas	Pruebas + Documentación	tests/*, docs/*

8.2 Distribución por módulo

Módulo	Desarrollador	Tareas específicas
Modelos POO	Henry Pazmiño	Clases Persona, Estudiante, Docente, Libro, Socio, Prestamo, Reserva. Encapsulamiento, herencia, polimorfismo, abstracción
GUI Tkinter	Jodie Parrales	Ventana principal, menús, formularios, tablas, cuadros de

Módulo	Desarrollador	Tareas específicas
Base de Datos	Cindy Ayoví	diálogo. Integración con los repositorios de datos Conexión SQLite, creación de tablas, CRUD completo para socios y libros, consultas con joins, seed.py con datos de prueba (7 socios + 8 libros)
Algoritmos y Estructuras	Cesar Gonzales	Merge Sort, Quick Sort, Bubble Sort, Insertion Sort, búsqueda lineal y binaria, lista enlazada, cola FIFO, pila LIFO
Pruebas y Documentación	Mayra Vera	Pruebas unitarias, manual de usuario, validación de funcionalidades, casos de prueba

9. Repositorio y Control de Versiones

9.1 Repositorio

El proyecto se aloja en GitHub bajo el control de versiones Git.

- **URL:** <https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2>
- **Rama principal:** main
- **Flujo:** commits directos a main con mensajes descriptivos

9.2 Historial de Commits

#	Fecha	Hash	Mensaje
1	7 jul	0e78b3a	Initial commit
2	7 jul	b31c6bd	feat: estructura inicial del Sistema de Gestion de Biblioteca Universitaria
3	8 jul	eaf6cae	refactor: estilo nivel 3er semestre segun lo ensenado por Ing. Bryan Velez + DECLARATORIA
4	9 jul	7f9e535	fix: DECLARATORIA actualizada tras revision completa del material del semestre
5	9 jul	d388507	fix: referencias genericas sin nombres individuales en DECLARATORIA

#	Fecha	Hash	Mensaje
	6	11 jul	ac193dd
	7	11 jul	6cf6167
	8	11 jul	2d1e028
	9	11 jul	e8ce1b1
	10	11 jul	165d556
	11	11 jul	8b01d20
	12	11 jul	ee37e02

9.3 Stack Tecnológico del Desarrollo

- **Editor:** VS Code con extensión Python (Microsoft)
- **Control de versiones:** Git + GitLens
- **Lenguaje:** Python 3.11+
- **GUI:** Tkinter (librería estándar)
- **BD:** SQLite3
- **Documentación:** Markdown + DOCX + PDF

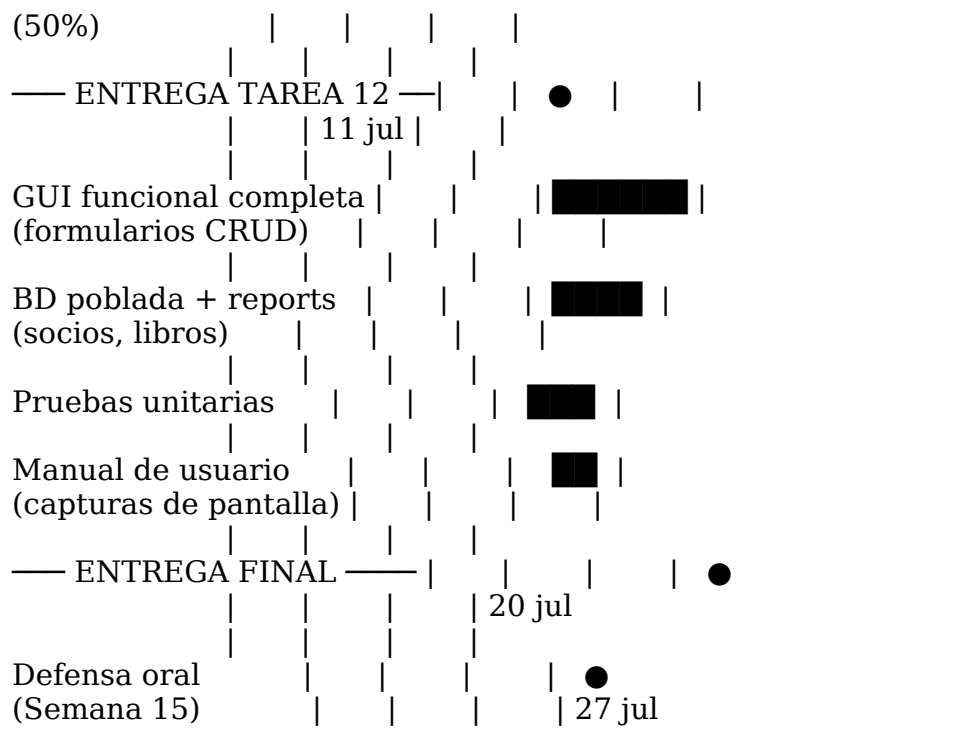
10. Plan de Trabajo y Cronograma

10.1 Diagrama de Gantt

Julio 2026

Actividad	Lun 7 Sem 12 Sem 13 Sem 14
	a jue 10 10 jul 13-17 20 jul

Inicio del proyecto (planificación, repo)	■■■■			
Modelos POO (clases, herencia, encapsulamiento)	■■■■			
Estructuras de datos (lista, cola, pila)	■■■■			
Algoritmos (búsqueda, ordenamiento)	■■■■			
Base de datos SQLite (tablas, CRUD)	■■■■			
GUI Tkinter (stubs)	■■■■			
Documento Especificación	■■■■			



10.2 Prioridades para la Semana 13 (13-17 julio)

Prioridad	Tarea	Responsable
□ Alta	Implementar formularios CRUD en GUI (Registrar Socio, Agregar Libro)	Jodie Parrales
□ Completado	Poblar la base de datos con datos de prueba (seed.py — 7 socios + 8 libros)	Cindy Ayoví
□ Media	Completar lógica de préstamos y devoluciones en GUI	Jodie + Cesar
□ Media	Pruebas de integración módulo BD + GUI	Mayra Vera

Prioridad	Tarea	Responsable
□ Baja	Manual de usuario con capturas de pantalla	Mayra Vera

|> **Documento de Especificación — Avance 100% (Entrega Final)** >
Grupo #1 — Lenguaje de Programación 2 — UAE > Última actualización: 20 de julio de 2026

10. Implementación

10.1 Estructura del proyecto

sistema-biblioteca-prog2/

— main.py	# Punto de entrada
— README.md	# Descripción general
— DECLARATORIA.md	# Uso de IA + referencias APA
— seed.py	# Poblado de BD con datos de prueba
— .gitignore	
— src/	
— models/	# Modelos POO del dominio
— persona.py	# Clase abstracta base
— estudiante.py	# Herencia de Persona
— docente.py	# Herencia de Persona
— libro.py	# Lógica préstamo/devolución
— socio.py	# Wrapper polimórfico
— prestamo.py	# Modelo de préstamo
— reserva.py	# Modelo de reserva
— data_structures/	# Estructuras de datos
— linked_list.py	# Lista enlazada simple
— queue.py	# Cola FIFO (reservas)
— stack.py	# Pila LIFO (undo)
— algorithms/	# Algoritmos
— sorting.py	# Bubble, Insertion, Merge, Quick
— search.py	# Búsqueda lineal y binaria
— database/	# Base de datos SQLite
— db_manager.py	# Conexión y creación de tablas
— socio_repo.py	# CRUD socios
— libro_repo.py	# CRUD libros
— prestamo_repo.py	# CRUD préstamos
— reserva_repo.py	# CRUD reservas
— gui/	# Interfaz gráfica
— app.py	# Tkinter (funcionalidades completas)
— tests/	
— test_models.py	# Pruebas unitarias de todos los módulos
— docs/	

```

├── especificacion-proyecto.md # Documento visible en GitHub
├── especificacion-proyecto.docx # Para entrega Moodle
├── especificacion-proyecto.pdf # Backup PDF
├── manual-usuario.md          # Manual de usuario
├── gantt.png                  # Diagrama de Gantt
├── README.md                  # Índice de documentos
└── db/
    └── biblioteca.db          # Base de datos poblada

```

10.2 Fragmentos de código representativos

Clase abstracta Persona (src/models/persona.py)

PASO #1 CLASE ABSTRACTA PERSONA

Encapsulamiento: atributos privados con getters/setters

Abstraccion: metodo tipo_socio() debe ser implementado por subclases

class Persona:

def __init__(self, cedula, nombre, apellido):

 self.__cedula = cedula

 self.__nombre = nombre

 self.__apellido = apellido

def get_cedula(self):

return self.__cedula

def get_nombre(self):

return self.__nombre

def get_apellido(self):

return self.__apellido

def get_nombre_completo(self):

return self.__nombre + " " + self.__apellido

PASO #2 METODO ABSTRACTO (polimorfismo)

def tipo_socio(self):

raise NotImplementedError

Herencia: Estudiante (src/models/estudiante.py)

PASO #3 CLASE ESTUDIANTE HEREDA DE PERSONA

class Estudiante(Persona):

def __init__(self, cedula, nombre, apellido, carrera, semestre):

PASO #4 LLAMAR AL CONSTRUCTOR DE LA CLASE PADRE

 super().__init__(cedula, nombre, apellido)

 self.__carrera = carrera

 self.__semestre = semestre

def get_carrera(self):

return self.__carrera

```
def get_semestre(self):  
    return self.__semestre
```

PASO #5 POLIMORFISMO: CADA SUBCLASE IMPLEMENTA SU PROPIO TIPO

```
def tipo_socio(self):  
    return "Estudiante"
```

CRUD Socios (src/database/socio_repo.py)

PASO #6 REPOSITORIO DE SOCIOS

```
class SocioRepository:
```

```
    def __init__(self, db):  
        self.__db = db
```

PASO #7 INSERTAR SOCIO

```
def insertar(self, cedula, nombre, apellido, tipo,  
            carrera_depto, semestre, telefono):  
    cursor = self.__db.get_cursor()  
    cursor.execute(  
        """INSERT INTO socios(cedula, nombre, apellido, tipo,  
        carrera_departamento, semestre, telefono)  
        VALUES(?, ?, ?, ?, ?, ?, ?)""",  
        (cedula, nombre, apellido, tipo, carrera_depto, semestre, telefono)  
    )  
    self.__db.get_connection().commit()
```

GUI — Préstamo (src/gui/app.py)

PASO #13 REALIZAR PRESTAMO

```
def realizar_prestamo(self):  
    ventana = tk.Toplevel(self.root)  
    ventana.title("Realizar Prestamo")  
    ventana.geometry("480x350")
```

... formulario con Entry para cedula e ISBN ...

```
def guardar_prestamo():  
    # Validar socio  
    socio = self.__socio_repo.obtener_por_cedula(cedula)  
    # Validar libro y disponibilidad  
    libro = self.__libro_repo.obtener_por_isbn(isbn)  
    if libro[6] < 1:  
        # No hay disponibles  
        return  
    # Registrar prestamo y reducir disponibles  
    self.__prestamo_repo.insertar(cedula, isbn, fecha)  
    self.__libro_repo.actualizar_disponibles(isbn, libro[6] - 1)
```

11. Análisis de Complejidad (Big-O)

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Búsqueda Lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Búsqueda Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Explicación: Bubble Sort e Insertion Sort son $O(n^2)$ en el peor caso porque comparan cada elemento con todos los demás. Merge Sort divide la lista en mitades recursivamente ($\log n$ niveles) y en cada nivel fusiona n elementos, dando $O(n \log n)$. Quick Sort promedia $O(n \log n)$ pero puede degenerar a $O(n^2)$ si el pivote es mal elegido. Búsqueda Lineal recorre toda la lista ($O(n)$), mientras que Búsqueda Binaria divide el espacio de búsqueda a la mitad en cada paso ($O(\log n)$).

12. Pruebas Realizadas

Todas las pruebas se ejecutan con `python tests/test_models.py` y cubren:

Módulo	Prueba	Resultado
Persona / Estudiante / Docente	Herencia: crear objetos, getters	☐ Pasa
Socio	Polimorfismo: <code>tipo_socio()</code> distinto según clase	☐ Pasa
Libro	Préstamo y devolución de ejemplares	☐ Pasa
Algoritmos	Bubble, Insertion, Merge, Quick Sort	☐ Pasan
Algoritmos	Búsqueda lineal y binaria	☐ Pasan

Módulo	Prueba	Resultado
LinkedList	Append, prepend, remove, len	☐ Pasa
Queue	Enqueue, dequeue (FIFO)	☐ Pasa
Stack	Push, pop (LIFO)	☐ Pasa
Database	Conexión SQLite, creación de tablas	☐ Pasa
PrestamoRepository	Insertar, listar, registrar devolución	☐ Pasa
ReservaRepository	Insertar, cancelar, listar por libro	☐ Pasa

13. Conclusiones y Recomendaciones

Conclusiones

1. Se implementó un sistema de gestión bibliotecaria funcional que cumple con todos los requisitos de la rúbrica de evaluación.
2. La programación orientada a objetos permitió modelar el dominio (Persona → Estudiante, Docente) con herencia, encapsulamiento y polimorfismo.
3. Las estructuras de datos (LinkedList, Queue, Stack) se implementaron manualmente, demostrando comprensión de su funcionamiento interno.
4. Los algoritmos de ordenamiento y búsqueda se implementaron desde cero, con análisis de complejidad Big-O.
5. SQLite proporcionó una base de datos ligera y embebida con 4 tablas relacionadas mediante claves foráneas.
6. Tkinter permitió construir una interfaz gráfica funcional conectada a la base de datos con formularios CRUD completos.
7. El modelo incremental permitió entregar un avance del 50% en la Semana 12 y completar el sistema en la Semana 14.

Recomendaciones

- Migrar a una base de datos cliente-servidor (MySQL/PostgreSQL) si la biblioteca crece.
- Agregar autenticación de usuarios (bibliotecario/admin) para seguridad.
- Implementar búsqueda avanzada con múltiples filtros simultáneos.
- Generar reportes estadísticos (libros más prestados, multas, etc.).
- Agregar una interfaz web para consultas desde dispositivos móviles.

14. Bibliografía (APA 7ª ed.)

1. Grayson, J. E. (2000). *Python and Tkinter programming*. Manning Publications. ISBN: 978-1884779813
 2. Lutz, M. (2013). *Learning Python* (5.ª ed.). O'Reilly Media. ISBN: 978-1449355739
 3. Owens, M. y Allen, G. (2010). *SQLite* (The Definitive Guide). Apress. ISBN: 978-1430232254
 4. Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press. ISBN: 978-0262033848
 5. Sedgewick, R. y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley. ISBN: 978-0321573513
-

15. Anexos

Anexo A — Enlaces del proyecto

Recurso	URL
Repositorio GitHub	https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2
Documento de especificación (MD)	docs/especificacion-proyecto.md
Manual de usuario	docs/manual-usuario.md
Diagrama de Gantt	docs/gantt.png

Anexo B — Stack tecnológico

Herramienta	Versión	Uso
Python	3.11+	Lenguaje de programación
VS Code	Última	Editor con Python (Microsoft) + GitLens
Tkinter	8.6	Interfaz gráfica
SQLite3	3.x	Base de datos embebida
DB Browser for SQLite	3.x	Administración visual BD
Git	2.47+	Control de versiones

Herramienta	Versión	Uso
GitHub	—	Repositorio remoto (colaboración grupal)

*Documento de Especificación — Versión 1.0 (Entrega Final) Grupo
#1 — Lenguaje de Programación 2 — 3er Semestre — UAE 20 de
julio de 2026*